

Stock Overview — Daily Stock Ledger Plan

KM Finny · planning document only (no code has been changed) · prepared 8 Oct 2026

The idea in one line: keep a stored table of each day's stock **movements** (purchase, sale, adjust) per plant and product. **Opening and closing are not stored — they are calculated** by adding up the days. This makes backdated orders safe and Stock Overview fast and easy to track.

1. How it works today

- Stock Overview keeps no daily table. On every request it works out opening, purchase, sale and closing from the raw records (`stock_movements` and `loading_scan_events`) with a heavy query (`/reports/plant-stock` in `server/routes/scan-sessions.ts`).
- A purchase is dated by the **order date**, not the scan day. Closing = Opening + Purchase + Adjust – Sale.
- The live count (`product_plant_stock.in_stock`) is what a scan checks and debits.
- Name, Sr No and SAP code are read live from Product Master each time.

2. The new table `stock_daily`

One row per **day + plant + product**. Every number is the movement for that day only.

Column	Meaning
<code>stock_date</code>	The day (India time)
<code>plant</code>	Canonical plant name (matched case-insensitively, as now)
<code>item_key</code>	Product id; if the barcode is not in Product Master, <code>bc: + the barcode</code>
<code>barcode</code>	The barcode on that day
<code>state</code>	The plant's state (GJ, MP, UP) from <code>plants.state</code>
<code>sr_no</code>	The cross-plant "New Sr" (works exactly as it does today)
<code>item_name</code>	Product Master name on that day
<code>sap_code</code>	The SAP code for that state (GJ SAP, MP SAP or UP SAP). If the plant has no state or the state has no SAP, the generic SAP code is used and <code>sap_is_fallback = true</code> flags the row.
<code>purchase_qty</code>	Boxes received (Scan Operations and Unloading), voids netted
<code>extra_qty</code>	The extra part of purchase (breakdown only)
<code>adjust_qty</code>	Corrections: manual Adjust, exchange, CSV edits, rollbacks of edits
<code>opening_entry_qty</code>	Opening Stock import (dated the day before, as now)
<code>system_adjust_qty</code>	Clear Stock and settings resets — shown, but kept out of the totals as now
<code>sale_qty</code>	What Loading loaded
<code>updated_at</code>	Last write

Key: (stock_date, plant, item_key). Index on (plant, item_key, stock_date).

Calculated, never stored

- **Day movement** = purchase + adjust + opening_entry – sale
- **Opening** of a period = sum of day movement for all days before it
- **Closing** = opening + the period's purchase + adjust – sale

3. Daily stock value (night snapshot)

- A night job (23:55 India time) writes the live count into `stock_day_snapshot` (day, plant, item, live_qty), copied from `product_plant_stock`.
- It is a record to track against. It is never used to calculate the screens.
- **Check:** for each day, calculated closing (plus system adjust) must equal the snapshot. Any difference is listed next morning, so a missed write is found quickly.

4. One way to write, so nothing is missed

- Today 36 places write to the stock ledger across 12 files: `loading.ts`, `unloading.ts`, `order-scan.ts`, `order-import-edit.ts`, `unloading-edit.ts`, `stockRecalc.ts`, `orderGroupReport.ts`, `settings-admin.ts`, `scan-sessions.ts`, `plant-stock-admin.ts`, `order-import.ts`, `opening-stock.ts`.
- Add one function, `postStockMovement`, that writes the ledger row **and** the daily row in the **same database transaction**. All 36 places call it, so the two can never disagree and a rollback undoes both.
- The daily row is written with an atomic increment: insert, and on conflict add to the existing amount.
- Name, SAP code and state are set when the row is first created that day.

5. What every action writes

Action	Day used	Plant	Column
Scan Operations scan (regular + extra)	Order's date from the CSV	Session plant	purchase +=, extra +=
Void of a scan	Same day as the scan	Same	purchase -=, extra -=
Unloading scan	Unloading batch's order date	Batch plant	purchase +=, extra +=
Unloading void	Same day	Same	purchase -=
Loading scan / Add Extra	Slip's order date	Each plant the boxes came from (state pool)	sale += each plant's part
Loading +/- adjust	Slip's order date	Pulled plant for +, slip plant for –	sale +=, -=
Loading void / quantity edit	Slip's order date	The pulled plants	sale -=
Delete from list (void or remove)	Slip's order date	The pulled plants	sale -= for every non-voided scan
Manual Adjust: add / remove / set	Today	Chosen plant	adjust += difference

Exchange between two products	Today	Plant	adjust -= on one, += on the other
Adjust Exchange Extra reassign	Today	Plant	adjust on both products
CSV edit / deleted CSV	CSV's order date	Plant	adjust ±
Opening Stock import	Day before the as-of date	Plant	opening_entry +=
Clear Stock, Remove All Operations Data	Today	Plant	system_adjust ±
Credit transfer between CSV parts	—	—	Nothing (no stock moves)

6. A worked day

Valsad, one GJ-SAP item, 7 Oct 2026. Stock before the day: 100.

- Scan 40 for an order dated 7 Oct → purchase = 40
- Void 5 → purchase = 35
- Load 30 for an order dated 7 Oct, taken 25 from Valsad and 5 from Vadodra → Valsad sale = 25, Vadodra sale = 5
- Manual adjust -2 → adjust = -2

Valsad row: opening 100, purchase 35, sale 25, adjust -2 → **closing = 108**, equal to the live count.

A later scan for an older order (3 Oct) lands in the 3 Oct row. Nothing else is touched; later opening and closing update themselves because they are summed.

7. How Stock Overview reads it

- **One date:** that day's row, plus the sum of all earlier days for opening.
- **A range:** opening = sum before the first day; purchase, adjust, sale = sums in the range; closing = the formula above.
- **No date:** from the Stock Tracking Start in Settings through today, as now.
- A simple sum over one table instead of today's multi-part query.

8. Rollout, step by step

1. **Create** `stock_daily` and `stock_day_snapshot` (schema change — needs approval).
2. **Rebuild history** from the existing ledger and Loading events, using today's rules (dates, plants, voids, pooling).
3. **Add** `postStockMovement`, then change the 36 write points one file at a time.
4. **Shadow mode** for 1–2 weeks: write to both, run the nightly check, compare the new Stock Overview with the current one.
5. **Switch** Stock Overview to the new table, keeping the old query behind a switch.
6. **Admin tools:** "Rebuild range" and a mismatch report.
7. **Tests** for every action in section 5, plus voids, pooled loads, backdated orders, a plant with no state, and a product rename mid-day.

9. Edge cases

- **Product not in Product Master:** keyed by barcode and flagged.
- **Barcode changed in Notion:** the same product id keeps its rows; old rows keep the old barcode text.
- **Rename (e.g. 60gm → 65gm):** a day's name is set when its row is first created. A same-day rename refreshes that day's row only; past days keep their old name.
- **SAP code changes in Notion:** past days keep the old code; the current day follows Product Master.
- **Two scans at once:** the existing row lock plus the atomic increment keep them safe.
- **Negative closing:** shown as now, never hidden.

10. Decisions still needed

1. Is a **night snapshot table** what is meant by "store the stock value daily", or is a stored closing number on each daily row also wanted? (Recommended: snapshot only — a stored closing is what makes backdated orders hard.)
2. Keep dating a purchase by the **order date** (as now)?
3. OK to rebuild history once from the old ledger and run old and new together before switching?
4. Approval for the schema change (two new tables) when work starts?